

Discovery Executive Summary

Project: discovery-002 · Generated: 15/07/2026, 16:21:54

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Components (F3), Business Logic in Components (F1), and Missing Frontend Service Layer (F2).

EXECUTIVE SUMMARY

This React-based multi-agent workflow UI exhibits significant architectural debt with oversized components, business logic embedded directly in views, and missing service/data abstraction layers. The most severe hotspots are god components exceeding 1400 LOC and widespread inline API calls scattered throughout the frontend. The dominant risk is change amplification — modifications to authentication, workflow management, or API contracts require changes across dozens of components due to tight coupling and lack of bounded contexts.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	167	MODERATE

H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	0	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	0	GOOD
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	0%	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	428	HIGH RISK
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	26	HIGH RISK
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	5	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	3	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	0	GOOD

1.4 Actions Required

Hotspot	Action	Rating	Priority
God / Oversized Components (F3)	Split 5 components >400 LOC into focused single-responsibility modules, starting with 3,214 LOC useAppStore.jsx	HIGH RISK	CRITICAL
Business Logic in Components (F1)	Extract workflow orchestration, agent lifecycle, and business rules from view components into service layer	HIGH RISK	CRITICAL
Missing Frontend Service/Data Layer (F2)	Centralize 26+ scattered API calls into domain-specific services with proper error handling and caching	HIGH RISK	CRITICAL
Fat Controllers (H1)	Extract business logic from server.js files into dedicated service classes before they exceed 300 LOC	MODERATE	MEDIUM

Prop Drilling / Global State (F4)	Implement granular contexts and reduce 3-level prop threading through intermediate components	MODERATE	MEDIUM
-----------------------------------	---	----------	--------

1.5 Expected Outcomes

- **Separation of Concerns:** Business logic extracted from view components enables independent testing and modification of domain rules without UI coupling
- **Maintainable Components:** Breaking down 1000+ LOC god components into focused modules reduces cognitive load and enables parallel development
- **Centralized API Management:** Consolidated data layer eliminates duplicate error handling and enables consistent offline/caching strategies across the application
- **Domain Boundaries:** Clear service abstractions allow teams to work on authentication, workflow management, and integrations independently
- **Change Resilience:** Reduced coupling between UI components and business logic minimizes change amplification when requirements evolve

The complete analysis has been saved to `docs/discovery/01-architecture-design.md` and covers both frontend and backend architectural concerns, with particular focus on the React frontend's significant architectural debt. The report identifies critical issues requiring immediate attention to improve maintainability and reduce change amplification risks.